Embedding Existing Pipelines in Design of CO_2 Transportation Networks for Optimal Sequestration Economics

David Nnamdi[1]* and R. G. Moghanloo[1]

[1]University of Oklahoma

Summary

In this paper, we present a method for incorporating existing pipelines into the design of optimal carbon dioxide (CO_2) transportation networks for carbon sequestration projects. The selection of the optimal pipeline transportation network is a crucial aspect of large-scale carbon sequestration projects as it greatly affects the project's economics. The method proposed in this paper aims to address the limitation of existing open-source tools, such as SimCCS$^{2.0}$, which are unable to accommodate existing pipelines in techno-economic optimization. With the recent amendment to the 45Q laws, which now offers 70% more tax credits for carbon sequestration than in 2018, energy companies are exploring the possibility of repurposing gas and liquid transportation lines for CO_2 transportation to abandoned oil and gas wells for carbon sequestration. This has further reinforced the need for a method that accounts for existing pipelines in sequestration economics.

The proposed method achieves this by representing the pipeline paths on the construction cost graph as zero-cost paths. Additionally, pipeline tie-in locations are fixed by creating exclusion zones that limit inflow edges around the pipeline path. The solution is then obtained by solving for candidate transportation network routes using graph shortest path algorithms. This reformulation of the CO_2 source-sink connection flow problem with limiting constraints on existing pipeline flow direction and capacity makes it possible to assess cost reduction associated with different CO_2 sources tie-in locations along existing transport pipelines.

The solution was developed using the Python programming language, and small-scale demo test cases have been used to illustrate its effectiveness in four tie-in cases with single pipeline and multiple pipelines that cut across several CO_2 source and sink locations. The method has also been applied to the evaluation of a proposed Carbon Storage Assurance Facility Enterprise (CarbonSAFE) II project and the results used to assess optimal pipeline tie-in points for expanded sequestration capacity.

The developed Python package is publicly available on GitHub for researchers and economic analysts to use for evaluating large-scale carbon capture, utilization, and storage projects, with the aim of encouraging further development and collaboration.

Introduction

In contemporary practice, the storage of CO_2 in deep saline aquifers is gaining prominence due to their immense potential for accommodating captured CO_2 (McPherson and Cole 2000). A primary obstacle in realizing large-scale CO_2 sequestration projects in saline aquifers is their economic feasibility. Unlike CO_2 enhanced oil recovery (EOR), which can counterbalance capture and storage expenses through augmented oil production, saline aquifer storage does not provide direct financial benefits.

Among the three principal expenses in CO_2 sequestration—injection, capture, and transportation costs—the former two are relatively well-understood. Injection costs can be precisely determined from historical data in the oil and gas sector, while capture costs, though site-specific, can be reasonably estimated at 11–75 US dollars per metric ton of CO_2 (USD/tCO_2) based on current technologies (Abramson et al. 2020). In contrast, transportation costs necessitate intricate analyses. Although CO_2 can be transported through various mediums, including rail, ships, trucks, and pipelines, pipelines are often indispensable for large-scale projects as they connect an extensive network of sources and sinks.

Determining the most cost-efficient pipeline route is crucial during the project's scoping phase, typically achieved through graph network optimization. Recent advancements have given rise to software tools such as SimCCS (Middleton and Bielicki 2009) and SimCCS$^{2.0}$ (Middleton et al. 2020). These tools compute optimal pipeline routes between multiple locations and estimate the corresponding transportation costs for a given volume of CO_2. Other approaches to optimizing CO_2 transport networks include the European Commission's JRC InfraCCS (Morbee et al. 2011) and the earlier GESTCO project (Gale et al. 2001), as well as multiperiod European planning models (e.g., CEPS, 2010). Nonetheless, the open-source SimCCS family (and SimCCS$^{2.0}$) has been widely adopted by academia, national laboratories, and industry and is commonly used as a baseline tool for carbon capture and storage (CCS) network design and scenario evaluation. However, a significant limitation of these tools is their inability to incorporate existing pipelines. Beyond SimCCS$^{2.0}$, recent work has expanded capabilities: SimCCS$^{3.0}$ introduced phase-based (multiperiod) planning (Ma et al. 2023); environmental/multiobjective routing frameworks now balance cost with ecological constraints (Shih et al. 2023); and geospatial "basin-splitting" improves routing fidelity around very large storage formations (Velasco-Lozano et al. 2024). In parallel, pipeline/right-of-way (ROW) reuse has been studied in newer applications (e.g., Ma et al. 2024), underscoring the need to evaluate how existing lines can be leveraged in network design.

Contribution and Novelty. With this paper, we introduce a workflow that explicitly embeds existing CO_2 pipelines into candidate-network construction and subsequent optimization by (i) encoding existing pipelines as zero-cost (or user-specified cost) arcs and (ii) adding a graph shortest-path layer that preconditions the network before mixed-integer linear programming (MILP) solution. Crucially, because we operate directly on the cost-surface graph, Sequestrix can deterministically control pipeline entry and egress using the four algorithms in Appendix A: (1) unrestricted ingress/egress, (2) fixed tie-in points, (3) permitted segment with exclusion zones, and (4) single tie-in with directional exclusion. These rules are enforced during candidate-network generation, so the MILP evaluates reuse

*Corresponding author; email: nnamdidavid.n@gmail.com

Copyright © 2025 Society of Petroleum Engineers

This paper (SPE 214917) was accepted for presentation at the SPE Annual Technical Conference and Exhibition, San Antonio, Texas, USA, 16–18 October 2023, and revised for publication. Original manuscript received for review 4 August 2025. Revised manuscript received for review 15 September 2025. Paper peer approved 30 September 2025.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

alongside new-build options under the exact operator constraints. This capability was not available in SimCCS$^{2.0}$ and has only recently been explored in parallel studies; our formulation makes it explicit, configurable, and reproducible.

As energy companies endeavor to fulfill environmental, social, and governance targets for CO_2 reduction, spurred by the enhancement of 45Q tax credits (26 U.S.C. § 45Q), there is an emerging interest in repurposing existing pipelines for CO_2 transport. Consequently, a methodology that accounts for the routes, capacities, and costs of existing pipelines is indispensable for the economic assessment of CO_2 sequestration projects. At the US deployment scale, the Department of Energy's (DOE) CarbonSAFE program and regional studies (e.g., Intermountain West/Four Corners) indicate that thousands of new pipeline miles may be required; leveraging existing CO_2 lines and ROWs can materially reduce greenfield construction and cost (DOE/NETL 2025 CarbonSAFE; Chen et al. 2022). We therefore emphasize methods that incorporate existing infrastructure in routing and optimization

45Q Tax Credit History and 2022 Adjustments. In 2008, the US government introduced Section 45Q of the Internal Revenue Code (26 U.S.C. § 45Q) to promote the expeditious adoption of carbon capture, utilization, and storage projects through carbon tax credits. However, by 2014, only 16 such projects had materialized, mainly attributed to the inadequate tax credits that were insufficient to balance operational costs (Wallace et al. 2015). The majority of these undertakings focused on CO_2 extraction at gas processing or fertilizer manufacturing facilities and its utilization in EOR operations.

In 2018, the government amended these tax credits, raising them to 35 USD/tCO_2 used in manufacturing or EOR, and 50 USD/tCO_2 for permanent geological storage. To qualify, operators had to adhere to strict criteria, including capturing at least 500,000 metric tons of CO_2 annually for power plants or 100,000 metric tons for other industries. Captured CO_2 had to be injected into underground geologic formations for permanent storage. For utilization in commercial products, with permanent storage demonstrated via life cycle analyses, the capture requirement was at least 25,000 metric tons of CO_2. While this revision made small-scale projects more viable, larger-scale projects remained economically challenging (DaneshFar et al. 2021).

In 2022, another revision was enacted, increasing tax credits by 71% for EOR and utilization in manufacturing to USD 60 per metric ton, and by 70% to USD 85 per metric ton for permanent underground sequestration. For direct air capture with permanent underground sequestration, credits of up to USD 180 per metric ton were introduced. Eligibility thresholds were also significantly reduced to 12,500 metric tons annually for point source capture, 18,750 metric tons for power plants, and 1,000 metric tons for direct air capture. This reform rendered potentially profitable many projects that were previously uneconomical. These policy changes help explain current interest in reusing existing corridors and assets to accelerate near-term deployments while controlling cost and siting risk (Congressional Research Service 2024).

Sequestration Planning and Optimization

Large-scale sequestration projects are capital intensive, requiring years to plan and implement. Time spent includes the project scoping phase when a suitable geologic aquifer has been identified and its storage capacity has been estimated. Following that, an assessment of the number of injection wells required to reach this capacity over a target duration must be determined, ensuring that injection pressure does not exceed fracture gradient and CO_2 plume stays within the defined geological confines (Hamzat et al. 2022). Assessing the number of injector wells gives a direct indication of the cost of storage, assuming historical well construction cost ranges from oil and gas CO_2-EOR injectors.

The next stage involves determining CO_2 emission sources which may either be industrial plants that have existing CO_2 separation and capture infrastructure as part of the process train (e.g., ammonia fertilizer or gas processing plants) or may require retrofitting new capture technology at flue gas emission points, generated from burning fossil fuels, to capture greenhouse gas emissions (e.g., in power plants). There are four basic systems for capturing CO_2, which are industrial process stream capture, post-combustion stream capture, oxy-fuel combustion capture, and precombustion capture (IPCC 2005). These systems have varying complexities and preferred application points leading to variability in estimating CO_2 unit capture costs. During the initial planning phase, one may associate high-level capture cost with plant type for economic simulations to identify best source-sink matches for defined geological aquifers.

Assuming pipelines as the primary option for transporting CO_2 from the capture facilities to the sinks, costing becomes a function of pipeline length and diameter. The length of the pipeline is related to the source-sink pair and the selected route which may be limited by physical barriers such as topography and residential settlements. Certainly, the longer the pipeline, the more costly it is (costs include acquiring ROW, pipeline materials, labor, equipment, and perhaps compressors to keep pressure within acceptable limits). The diameter of the pipeline is related to the amount of CO_2 it will be transporting and the inlet/outlet pressure requirements. Economics of scale is possible with larger pipelines and constructing large trunk lines for CO_2 transport may be optimal in certain cases (Yaw et al. 2019). In this work, we encode existing CO_2 pipelines directly in the candidate network (as zero-cost arcs) and apply entry/egress algorithms so that routing and subsequent MILP optimization can reuse known alignments under operator-specified tie-in and exclusion rules.

Optimizing Source-Sink Matching. While capture and storage costs can only be minimized by technological improvements as the CCS industry matures, the science of pipeline construction is fully mature, and cost improvements are majorly a function of defined routes and spatial trajectories. Optimizing source-sink matching and transport route generation has been the focus of a subset of discrete mathematics, operations research, and geographical information systems (GIS). Geo-spatial coordinates can be represented using graph theory and least cost paths (LCPs) generated between node pairs, given a cost surface that maps out cost of moving (or moving an element) from one node point to another. To effectively optimize pipeline deployment, a network of LCP (candidate network) must be generated for a given set of sources and sinks, and then formulated as a MILP problem. This is elaborated on in subsequent sections. Recent advances add temporal phasing (Ma et al. 2023), environmental trade-offs (Shih et al. 2023), and improved sink granularity (Velasco-Lozano et al. 2024); our contribution is complementary—embedding existing pipelines during candidate network generation so that the MILP evaluates reuse alongside new-build options.

Capturing Existing Pipeline in Candidate Network Generation

Cost Surface Map: Generation and Representation. In 2019, a joint research initiative by Los Alamos National Laboratory and Montana State University gave rise to an open-source software named Cost Surface Multi-Layer Aggregation Program (CostMAP). This tool was designed to generate rasterized cost surfaces, leveraging pre-existing GIS data, including land cover, slope, population density, and data on natural and man-made barriers as well as pipeline networks (Hoover et al. 2020).

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

CostMAP provides an advanced method for calculating edge weights, particularly in scenarios with barriers like rivers or roads. The software employs a high-resolution raster to meticulously examine cells for genuine crossings or corridors, rather than broadly classifying a cell as a barrier or crossing based on partial feature presence. This refined approach helps to avoid overestimating the costs associated with barriers and identifies feasible routes that may be overlooked with other methods. By deploying major and minor cells, CostMAP optimizes the identification of actual crossings between neighboring cells, thereby improving the accuracy of edge weight estimations in the presence of barriers. The software's output, an aggregated cost raster and a weighted-cost network, can be used for LCP calculations in any GIS-aware software.

In 2022, the release of an updated version, CostMAP^{PRO}, aimed to overcome scaling issues associated with large-scale sequestration projects (Talsma et al. 2022). CostMAP^{PRO} produces high-resolution routing surfaces (from 90 m to 720 m, with 10-m modeling capability), separates routing and construction cost weights for more flexibility, and allows users to enter custom weights to develop custom cost surfaces. The latter feature improves the tool's adaptability to specific stakeholder requirements, such as the need to avoid river crossings.

Capturing Existing Pipeline Geolocation. Pipeline operators possess precise latitude-longitude coordinates of the pipelines they construct and manage, and this information, often stored as shape files or other GIS formats, is mandatorily submitted to governmental regulators for recordkeeping and assistance in ROW assessments and potential encroachments for new pipeline constructions. The US maintains a comprehensive database, known as the National Pipeline Mapping System (NPMS), encompassing all pipelines transporting liquids, gases, and hazardous waste. Although the complete data set is not openly accessible, researchers can employ the NPMS Public Viewer tool to meticulously trace the contours of pipelines pertinent to CO_2 sequestration optimization when data from operators is not available. **Fig. 1**. exemplifies the use of the NPMS Public Viewer, capturing an image of all pipelines in Osage County, Oklahoma, USA (https://pvnpms.phmsa.dot.gov/PublicViewer/).

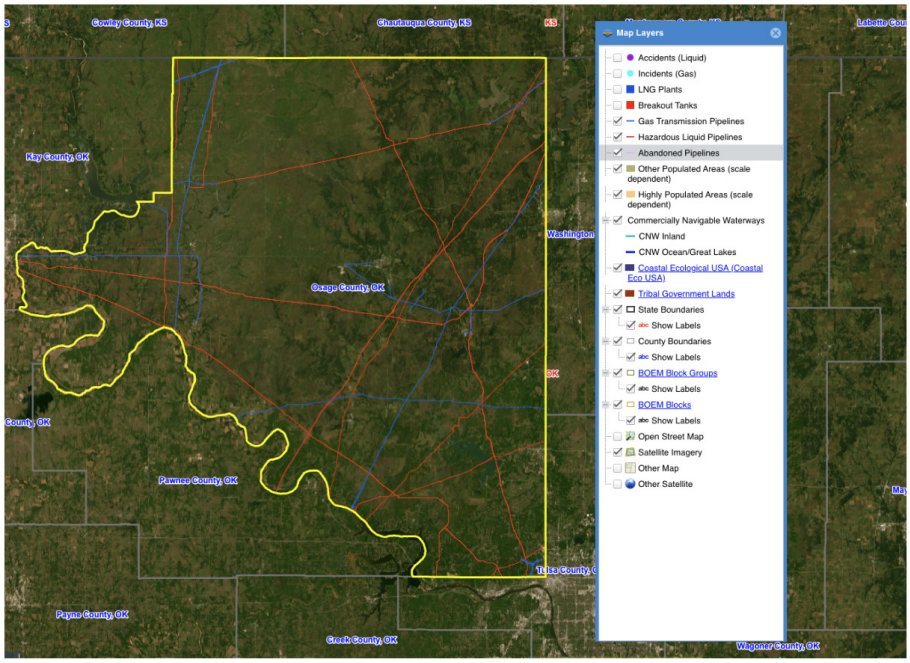

Fig. 1—Pipelines for Osage County (PHMSA, NPMS Public Viewer, accessed 27 April 2023).

Embedding Existing Pipelines in the Cost Surface Map. When transporting CO_2, costs are contingent upon the chosen transportation route, which is selected based on LCPs on a surface cost graph. An intuitive but challenging solution is to modify the cost graph by setting the weights of edges along an existing pipeline route to zero. To facilitate this, several assumptions are made:

1. Long transport pipelines comprise discrete shorter fixed-length pipelines connected by fittings such as elbows, tee connections, and reducers.
2. These shorter pipelines are connected via welding or coupling using fittings.
3. Over long distances, the length of the pipe fittings can be disregarded; only the length of the short, fixed pipelines is considered.
4. Each fixed pipeline's ends represent actual physical locations with latitude and longitude, allowing the pipeline's representation as a discrete set of Lat-Long points.

These assumptions permit the direct modification of the edges representing the existing pipeline by setting their weights to zero. However, this necessitates the exact edges to be present on the surface cost graph, which is often not feasible due to several reasons:

- The shorter pipelines have a fixed length, but due to the Earth's curvature, distances between consecutive grid points may vary, causing a mismatch between pipeline Lat-Long and grid Lat-Long.
- One might contemplate generating a high-precision cost graph by decreasing grid spacing. However, this approach becomes impractical due to the limitations in graph generation and shortest distance calculations. As the grid spacing diminishes, the number of vertices and edges escalates, leading to a quadratic increase in computational time. Data storage also becomes a significant concern for finer grids.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

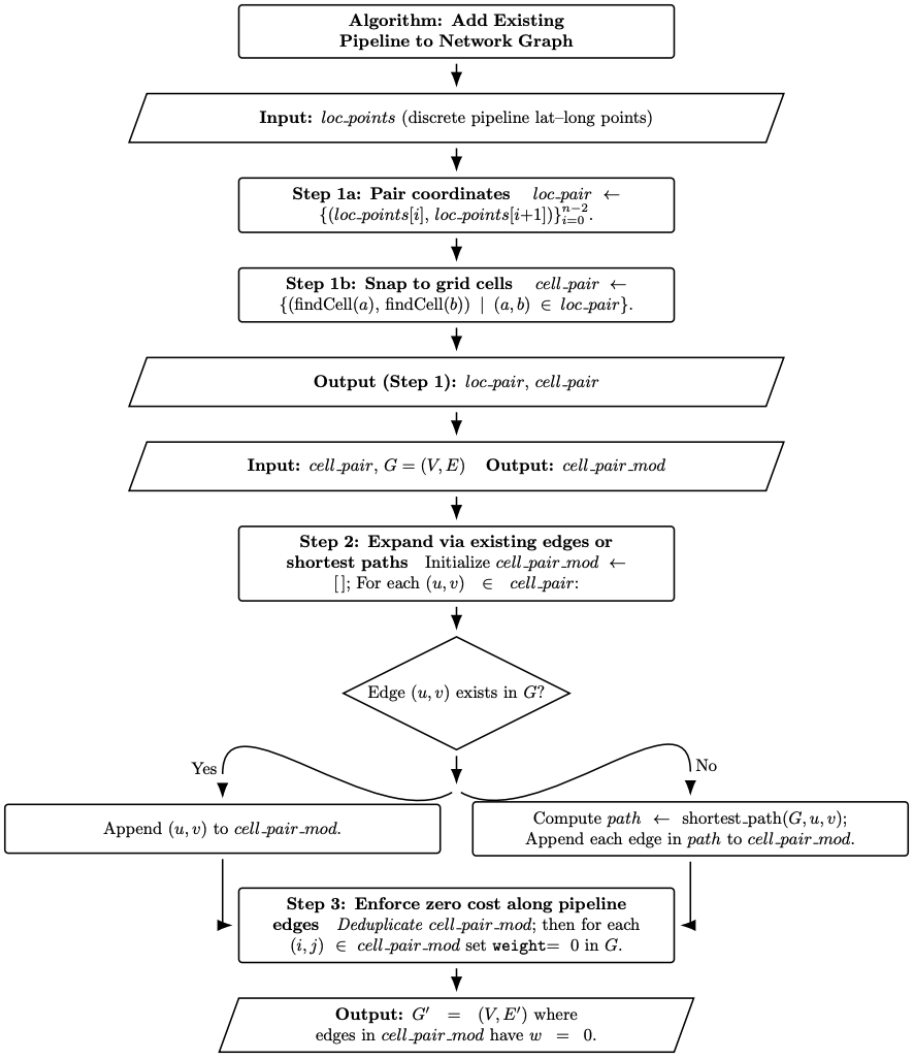

Fig. 2—Flow chart of algorithm to add existing pipeline to network graph.

Considering these limitations and the fact that most operators or researchers lack an exact set of discrete Lat-Long points for all pipe segments in a pipeline network, a more pragmatic solution was developed:

1. Sparse Lat-Long coordinates representing sections along the pipeline are obtained, either from the operator or manually using the US NPMS. Each section is assumed linear and can be connected by a straight line.
2. Using the geolocations of these sections, approximate x and y grid coordinates are calculated.
3. If an edge exists between successive geolocation grid points on the map, the weight of the edge is set to zero. If an edge does not exist, some are formulated.
4. Formulation involves calculating the shortest paths between two grid points on the graph using weighted edges. Once the grid points and edges along these shortest paths are identified, they are added to the set of pipeline Lat-Long points, and the edge weights are set to zero.

The implementation of this approach is demonstrated in the algorithm presented below **(Algorithm 1)**, and a flow chart to illustrate execution sequence is shown in **Fig. 2**.

Controlling Entry and Exit Nodes of Existing Pipeline. After altering the cost surface graph by incorporating zero-weight edges representing existing pipelines, a crucial concern emerges regarding the control of entry and exit points from these pipelines. This issue arises primarily due to the existence of a zero-weight path in the graph. When calculating the shortest path between any two locations in proximity to the pipeline, multiple entry and exit points may emerge surrounding the pipeline, as demonstrated in **Fig. 3**. Specifically, **Fig. 3** illustrates how modeling an existing pipeline as a zero-cost corridor changes Dijkstra's LCPs between two location pairs, (10→20) and (2→12). **Fig. 3a** shows the baseline LCPs without a pipeline (blue). **Fig. 3b** introduces the pipeline as a zero-weight corridor (green). **Fig. 3c** shows the updated LCPs (purple): The portions that overlap the green corridor represent travel along the zero-cost pipeline, while the purple segments outside the corridor are the positive-cost connectors used to enter and exit the pipeline. This highlights that multiple admissible entry and exit nodes can arise along a single pipeline, which motivates explicit control of entry/exit behavior.

In real-world applications, it is often undesirable to have multiple entry and exit points along a single pipeline due to pressure considerations. Maintaining or regulating pressure at entry points is essential to prevent backflow of fluids from higher to lower pressure points. This necessitates the installation of compressors and pressure monitoring instruments, which can be costly.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

Algorithm 1—Add an existing pipeline to the network graph.

```
Step 1: Convert discrete pipeline lat-long points to edges
Input: loc_points, a set of discrete pipeline lat-long points
Output: loc_pair, a list of edge pairs
   Create a new variable: loc_pair = list()
   for i in range(len(loc_points) – 1):
      loc_pair.append((loc_points[i], loc_points[i+1]))
   end for
   Find cell location of loc_pair points and assign to cell_pair using findCell function
   for i in range(len(loc_pair)):
      cell1 = findCell(loc_pair[i][0])
      cell2 = findCell(loc_pair[i][1])
      cell_pair.append((cell1, cell2))
   end for
Step 2: Modify cell_pair to include edges from shortest paths algorithm
Input: cell_pair, a list of edge pairs; edges, a set of edges in G(V, E)
Output: cell_pair_mod, a modified list of edge pairs
   Create a new variable: cell_pair_mod = list()
   forall nodepair in cell_pair:
      if nodepair in edges:
      cell_pair_mod.append(nodepair)
   else:
      path = shortest_path(source=nodepair[0], destination=nodepair[1])
      forall edge in path:
         cell_pair_mod.append(edge)
      end for
      end if
end for
Step 3: Modify graph G(V, E) and set all edge weights for edges in cell_pair_mod to zero
Input: G(V, E), a cost surface graph; cell_pair_mod, a modified list of edge pairs
Output: G'(V, E'), a modified cost surface graph with some edge weights set to zero
   forall nodepair in cell_pair_mod:
      G.edges[nodepair][weight] = 0
   end for
```

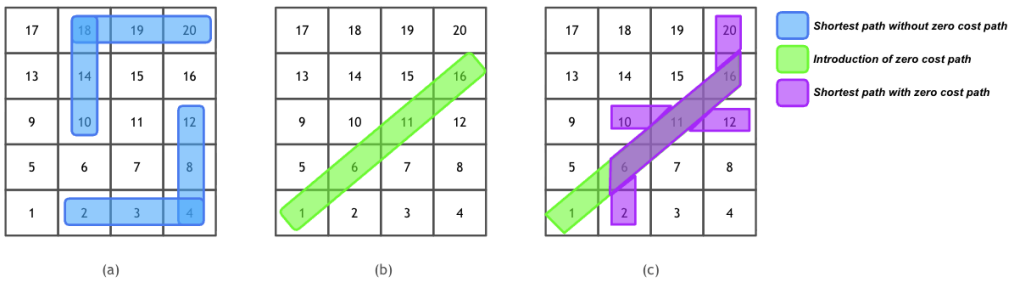

Fig. 3—Effect of a zero-cost pipeline on LCPs. (a) Baseline shortest paths without a pipeline (blue). (b) Existing pipeline encoded as a zero-weight corridor (green). (c) Resulting shortest paths when the corridor is available (purple): Overlap with green indicates movement along the zero-cost pipeline; nonoverlapping purple segments are positive-cost connectors from the origin to the pipeline (entry) and from the pipeline to the destination (exit).

Assuming an operator opts for a single entry point on an existing pipeline to allow CO_2 from nearby capture sites and is amenable to multiple exit points for routing CO_2 to storage sites, a pertinent question arises: Should the entry points be chosen based on factors such as regional knowledge, land agreements, and company preferences, or should they be determined through shortest path calculations?

In addressing this issue, four scenarios for implementing entry points and "exclusion" zones (areas along the pipeline where tie-ins are not permitted, which may be vital for sections through residential or hilly areas) should be considered, as depicted in **Fig. 4**:

- **Case a:** Two tie-in points (entry and exit) along a given pipeline, with exclusion zones in all other areas.
- **Case b:** Two tie-in points, but with exclusion zones at the ends of the pipeline.
- **Case c:** A single tie-in point and exclusion zones everywhere except at the source or sink.
- **Case d:** A single tie-in point with exclusion zones either before or after the tie-in point.

These cases help in strategically selecting entry and exit points while considering operational constraints and preferences. Algorithms to implement these cases are presented in Appendix A.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

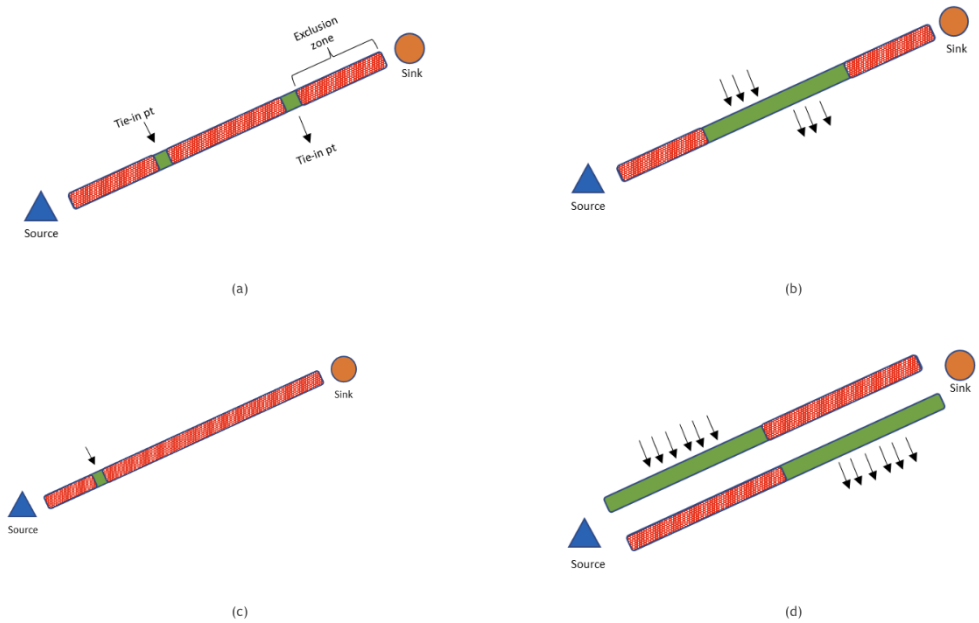

Fig. 4—Illustration of the four methods to define tie-in points along an existing pipeline. (a) Two tie-in points are explicitly defined with exclusion everywhere else in pipe. (b) Two tie-in points with exclusion at ends. (c) Single tie-in point with exclusion everywhere else but source/sink. (d1) Single tie-in point with exclusion after. (d2) Single tie-in point with exclusion before.

Candidate Network Generation. The process of generating alternate pipeline transport networks, or candidate networks, aims to create a network of LCPs that connect sources to sinks, ensuring that a path connects each source to every sink. As outlined by Middleton et al. (2012), there are five primary steps in creating these networks:

1. The creation or importation of a cost surface, possibly using tools like CostMAP.
2. Extraction of raster LCPs using shortest path algorithms such as that in Dijkstra (1959).
3. Conversion of raster to vector, effectively simplifying LCPs into discrete nodes and arcs.
4. Removal of redundancies by eliminating duplicate edges along identical cost paths
5. Refinement of the network using algorithms designed to further simplify the network, such as by merging nodes or collapsing triangles,

When connecting sources and sinks on a large graph with multiple edges, numerous routes may exist, which can lead to intractability or computational inefficiency when trying to determine routes between several pairs of nodes. A solution to this challenge was proposed by Yaw et al. (2019) in their research on efficient network design. They introduced the Greedy Subset Spanner (GSS) algorithm as an optimal design solution for pipeline transport routes. Their findings showed that the GSS algorithm could reduce the number of edges in a base graph by more than 99.9% while only increasing the cost of alternate routes by approximately 6% compared with the most optimal base Steiner trees.

Additionally, Delaunay triangulation (DT) has been effectively used to generate a set of node pairs for which cost paths can be determined (Delaunay 1934). However, due to the geospatial nature of pipeline network generation, the distance guarantee of no more than 2.418 times the Euclidean distance between any two points is not maintained (Keil and Gutwin 1992). Despite this limitation, the GSS algorithm and DT both demonstrated significant edge reduction, with DT yielding better costs. By integrating DT into the second step of the outlined five-step workflow, optimal candidate networks can be generated.

Introducing Sequestrix

We introduce the Sequestrix CO_2 Network Optimization Tool, designed as an open-source packaged web interface to perform CO_2 sequestration project economic analysis considering existing pipelines with predefined tie-in points and transport cost estimates. Open-sourcing Sequestrix is primarily done to encourage large-scale sequestration research and project planning, which will, in turn, lead to accelerated implementation and bring the US and rest of the world closer to net-zero goals.

Sequestrix Optimization Framework. The Sequestrix framework, as illustrated in **Fig. 5**, takes four major input data categories, which are:

- CostMAP weighted cost surface graph which forms the basis for determining cost-optimal new pipeline routes while building the candidate network.
- CO_2 source data from emitting plants and direct air capture. Geographic location, CO_2 capture capacity in metric tons per year, and capture costs in USD/tCO_2 are required as part of this input data category.
- CO_2 sink data (primarily taken from geologic saline aquifers) are also required. Geographical location, storage capacity in metric tons, and injection costs in USD/tCO_2 are the input parameters. Note that other utilization pathways, such as EOR or chemical products, may be captured here with the similar input parameters.
- Existing pipeline Lat-Long sequences from shape files given by pipeline operators or from web scraped data in NPMS with associated capacities, transport/build costs, and tie-in points are optional requirements.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

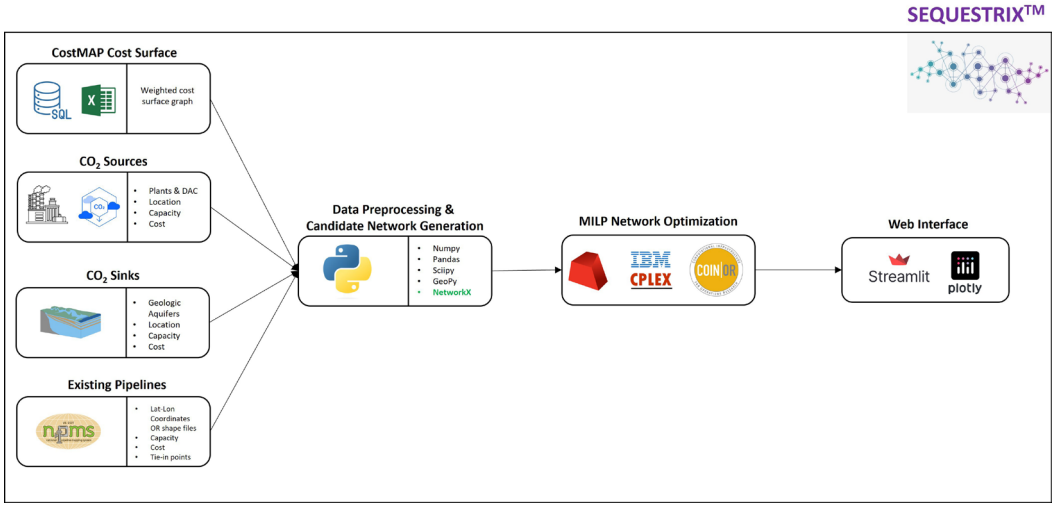

Fig. 5—Sequestrix CO_2 sequestration optimization tool framework starting from data sources to preprocessing, optimization, and visualization in web interface (left to right). DAC = direct air capture.

In the data preprocessing and candidate network generation phase, Python data manipulation, spatial, and graph network analysis packages are used to transform geolocations to vertices on the cost surface graph, generate DT, embed existing zero-cost paths in the cost surface, and find the LCPs for transporting CO_2. Subsequently, a mathematical representation of the candidate network coupled with cost information (capture, injection, and transport) is used to run an MILP optimization scheme to find the best source-sink pairings, with the best transportation route to sequester a target amount of CO_2.

The results of the network optimization are displayed on a Streamlit-powered web application with key cost and transport metrics displayed for visual evaluation using graphical objects from the Plotly Python library. Multiple scenarios for various sources, sinks, existing pipelines, or price points may be run and compared within Sequestrix using a proprietary result storage format. The solution is containerized using Docker and deployed to the cloud for multiple user access. Details on the implementation can be found on the GitHub repository where the code is hosted: *https://github.com/davidpcg01/CO_2-TRANSPORT-NETWORK-OPTIMIZATION-PROJECT*.

Mathematical Representation of Sequestration Optimization. Converting the described elements into mathematical language for the purpose of creating a CO_2 optimization model is made uncomplicated due to an existing framework formulated and put into operation within SimCCS as referenced in Middleton et al. (2020). The mathematical model that follows employs notations derived from SimCCS (see also the Nomenclature).

The model formulation is a MILP problem, and the objective function as described in the previous section is to minimize sequestration cost. Sequestration costs consist of capture cost, storage cost, pipeline build cost, and transportation cost, written mathematically as

$$\min\left[\underbrace{\sum_{i\in S}\left(F_i^S s_i + V_i^S a_i\right)}_{capture} + \underbrace{\sum_{j\in R}\left(F_j^R r_j + V_j^R b_j\right)}_{storage} + \underbrace{\sum_{a\in A}\sum_{c\in C}\beta_{ac} y_{ac}}_{pipe\ build} + \underbrace{\sum_{a\in A}\sum_{c\in C}\alpha_{ac} f_{ac}}_{transportation}\right],$$

and subject to the following constraints:

- Arc capacity bounds: $Q_{ac}^{min} \le f_{ac} \le Q_{ac}^{max} \quad \forall\, a \in A,\ \forall\, c \in C.$
- Single direction arc flow: $\sum_{c\in C} y_{ac} \le 1 \quad \forall\, a \in A.$
- Flow balance: $\sum_{\substack{a\in A\\ src(k)=n}}\sum_{c\in C} f_{ac} - \sum_{\substack{a\in A\\ dst(k)=n}}\sum_{c\in C} f_{ac} = 0 \quad \text{if } n \in N.$
- Demand balance: $\sum_{\substack{a\in A\\ src(k)=n}}\sum_{c\in C} f_{ac} - \sum_{\substack{a\in A\\ dst(k)=n}}\sum_{c\in C} f_{ac} = -b_n \quad \text{if } n \in R.$
- Supply balance: $\sum_{\substack{a\in A\\ src(k)=n}}\sum_{c\in C} f_{ac} - \sum_{\substack{a\in A\\ dst(k)=n}}\sum_{c\in C} f_{ac} = a_n \quad \text{if } n \in S.$
- Capture capacity bounds: $a_i \le Q_i^S s_i \quad \forall_i \in S.$
- Storage capacity bounds: $b_j \le Q_j^R r_j \quad \forall_j \in R.$
- Target capture: $\sum_{i\in S} a_i \ge CO_2 T.$
- Pipeline build cost: $C_1 * f_{ac} + C_2 = \alpha_{ac}\ \forall_a \in A$, where C_1 and C_2 are trendline constants.
- Convergence criteria: Full solver termination settings (relative Mixed Integer Programming (MIP) gap feasibility tolerances, and time limits) are documented in Appendix B.

The constants $C_1 \in C$ and $C_2 \in C$ are based on trend lines used to represent cost per unit distance profiles for given CO_2 volume. Representing CO_2 transportation costs with linearized trends was proposed in Middleton (2013), where pipeline transportation costs were a function of the volume of CO_2 being transported. In other publications (Jones et al. 2022; Whitman et al. 2022), researchers demonstrated the use of two linear trends that cover 11 distinct pipeline capacities in SimCCS.

The base optimization equation stays relatively similar; however, implementation differs between Sequestrix and SimCCS. In Sequestrix, the cost of transportation (pipeline build cost) along existing pipeline can be set to zero or to a user-specified input. The capacity Q_{ac}^{max} along an existing pipeline is also user-specified. For all other pipeline arcs, transport cost per arc is modeled as a piecewise-linear function of annual CO_2 flow, consistent with prior SimCCS studies. In this representation, **pipeline diameter is not an explicit decision**

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

variable and is therefore not reported; the diameter/compression choices are implicitly embedded in the cost-flow trendlines used for the optimization.

Recovering per-segment diameters from solved flows—e.g., by external cost tools consistent with the trendlines or by standard CO_2 pipeline hydraulics—will be addressed as future work and is outside the scope of Sequestrix.

Enid-Purdy Case Study

In a report published by the US DOE and the National Energy Technology Laboratory (NETL) (Wallace et al. 2015), it was documented that the midcontinent region of the United States, specifically in Oklahoma (OK) and lower Kansas (KS), contains five major pipelines dedicated to the supply of CO_2 from industrial sources to operators in the oil and gas industry for CO_2-EOR. These pipelines are visually represented in **Fig. 6**, while **Table 1** comprises information regarding their operators, lengths, and capacities. For the purpose of demonstrating Sequestrix's capability in assimilating existing pipelines into network optimization, the Enid-Purdy Pipeline, situated in central OK, was chosen.

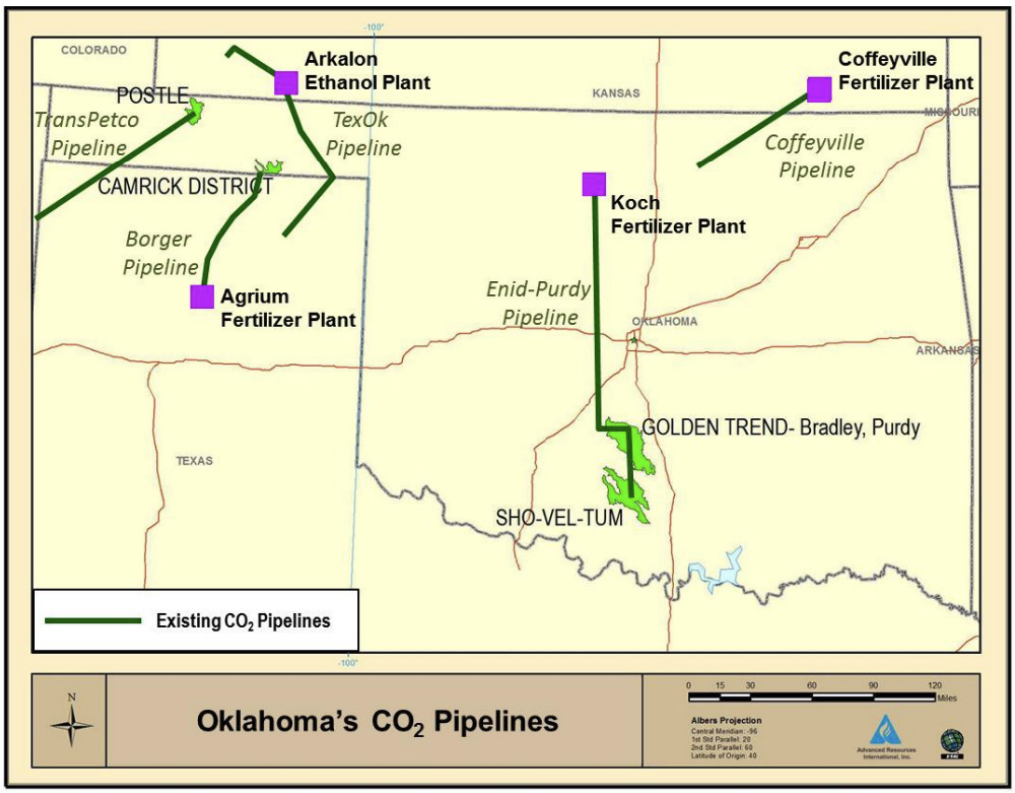

Fig. 6—Mid-Continent CO_2 pipeline infrastructure spanning OK and lower KS [from Wallace et al. 2015, Exhibit 9, p. 11). Image courtesy of the US DOE, NETL.

Scale	Pipeline	Operator	Location	Length (miles)	Diameter (in.)	Estimated Flow Capacity (MMscf/D)
Small-scale distribution systems	Coffeyville-Burbank	Chaparral Energy	KS, OK	68	8	80
	Enid-Purdy (Central OK)	Anadarko	OK	117	8	80
	TransPetco	TransPetco	TX, OK	110	8	80
	TexOk	Chaparral Energy	OK	95	6	70
	Borger	Chaparral Energy	TX, OK	86	4	50

KS = Kansas; OK = Oklahoma; TX = Texas

Table 1—Ownership details and specifications of midcontinent transport pipelines (Wallace et al. 2015).

The selection was motivated by the pipeline's strategic location, which is in close proximity to various CO_2 sources and sinks, thereby enabling a comprehensive analysis of diverse scenarios through Sequestrix.

Data Set Description. A detailed list of sources, sinks, their geolocations, and capacities is given in **Tables 2 and 3**.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

ID	Unique Name	**Capture** Capacity ($MTCO_2$/yr)	Total Unit Cost (USD/ tCO_2)	Lat.	Long.
1	Oxbow Calcining LLC	0.32	56	36.545	–97.85
2	Mustang	0.53	75	35.471	–97.673
3	Wynnewood Refining Co.	0.63	75	34.629	–97.169
4	Redbud Power Plant	2.3	75	35.685	–97.224
5	Horseshoe Lake	0.6	75	35.509	–97.179
6	Cana Gas Plant	0.1	14	35.535	–98.099
7	OHL NGLP Medford Plant	0.1	14	36.776	–97.756
8	Terra International (Oklahoma) Inc.	0.2	17	36.437	–99.471

$MTCO_2$/yr = million tonnes of CO_2 per year

Table 2—CO_2 sources for Enid-Purdy study.

ID	Unique Name	Storage Capacity ($MTCO_2$)	Total Unit Cost (USD/ tCO_2)	Lat.	Long.
1	Field Outlet 1	4	–31	34.381	–97.749
2	Purdy Field	3.4	–31	34.758	–97.605
3	i1-i3	0.33	–31	35.184	–98.201
4	i10	0.12	–31	34.482	–97.707
5	i11	0.12	–31	34.506	–97.598
6	Cluster 6	3.24	–31	34.473	–97.441
7	Cluster 7	2.49	–31	34.421	–97.614
8	Cluster 4	1.8	–31	35.3	–98.295

$MTCO_2$/yr = million tonnes of CO_2 per year

Table 3—CO_2 sinks or storage sites for Enid-Purdy study.

In summary, there are eight sources with a combined annual capture capacity of 4.77 million tonnes of CO_2/year ($MTCO_2$/yr) and eight sinks with a combined storage capacity of 15.5 $MTCO_2$. Unit storage cost of −31 USD/tCO_2 was assigned based on the assumption of injection costs of 4 USD/tCO_2 with the 2018 tax credit of 35 USD/tCO_2, which accounts for storage in geologic reservoirs via CO_2-EOR. The Enid-Purdy pipeline is captured using geolocation points obtained from NPMS and is given a transport capacity of 2 $MTCO_2$/yr.

We evaluate five scenarios to assess entry/exit controls in Sequestrix:

- **Base case:** No embedded existing pipeline. Serves as the reference for Scenarios a–d.
- **Scenario a: Enid Purdy Pipeline with 2 $MTCO_2$/yr Cap, No Tie-in Points, No Exclusion—**Ingress and egress are allowed on any point along the pipeline, with the aim of minimizing transport costs. This scenario is used to assess Case 1, as shown in **Fig. 3a**.
- **Scenario b: Enid Purdy Pipeline with 2 $MTCO_2$/yr Cap, 2 Tie-in Points, No Exclusion—**This scenario more accurately reflects real-world circumstances, as there are often limitations on the number of tie-in locations an operator is willing to accommodate along a pipeline, which may be due to topographical, environmental, or land rights constraints. The specified tie-in locations are Lat 35° 57′ 1.7″ N, Long −97° 47′ 30.58″ W, in the northern section of the pipeline, and Lat 35° 0′ 42.03″ N, Long −97° 46′ 52.54″ W, in the middle to lower pipeline section. This illustrates Case a **(Fig. 4a)**.
- **Scenario c: Enid Purdy Pipeline with 2 $MTCO_2$/yr Cap, 2 Tie-in Points, Exclusion at Ends—**In this instance, the operator delineates a region for permissible tie-ins through specifying two geographic coordinates, which is in reference to Case b **(Fig. 4b)**. The coordinates chosen for Tie-In Point 1 are Lat 36° 18′ 53.71″ N, Long −97° 46′ 58.06″ W, while for Tie-In Point 2, they are Lat 36° 39′ 27.32″ N, Long −97° 46′ 58.06″ W. This illustrates Case c **(Fig. 3c)**.
- **Scenario d: Enid Purdy Pipeline with 2 $MTCO_2$/yr Cap, Single Tie-in Point with Exclusion Before—**In this instance, the operator has specified a single point along the pipeline where tie-ins are permissible, beyond which no tie-in is allowed. The geolocation for this tie-in point is Lat 36° 40′ 27.42″ N and Long −97° 47′ 2.04″ W. This illustrates Case d **(Fig. 4d)**.

In all five scenarios, the project duration is set at 10 years with a target capture amount of 1.55 $MTCO_2$/yr and a capital recovery factor of 10% on the pipeline construction cost. The results of these scenarios are compared both qualitatively **(Figs. 7 and 8)** and quantitatively **(Figs. 9 and 10)**.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

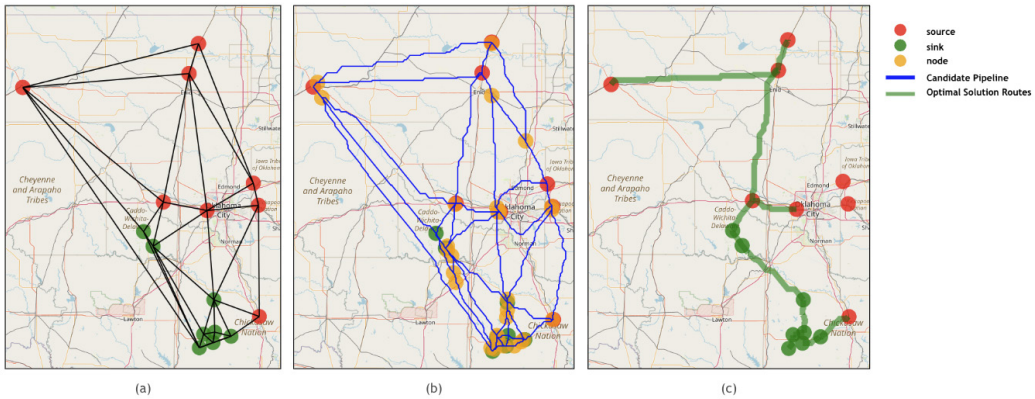

Fig. 7—Sequestrix base case solution without embedding the Enid-Purdy pipeline. (a) DT connecting all locations (red = sources; green = sinks; orange = junction nodes). (b) Candidate pipeline corridors shown as thin blue polylines over the triangulation. (c) Optimal pipeline routes selected by the model shown as solid green polylines.

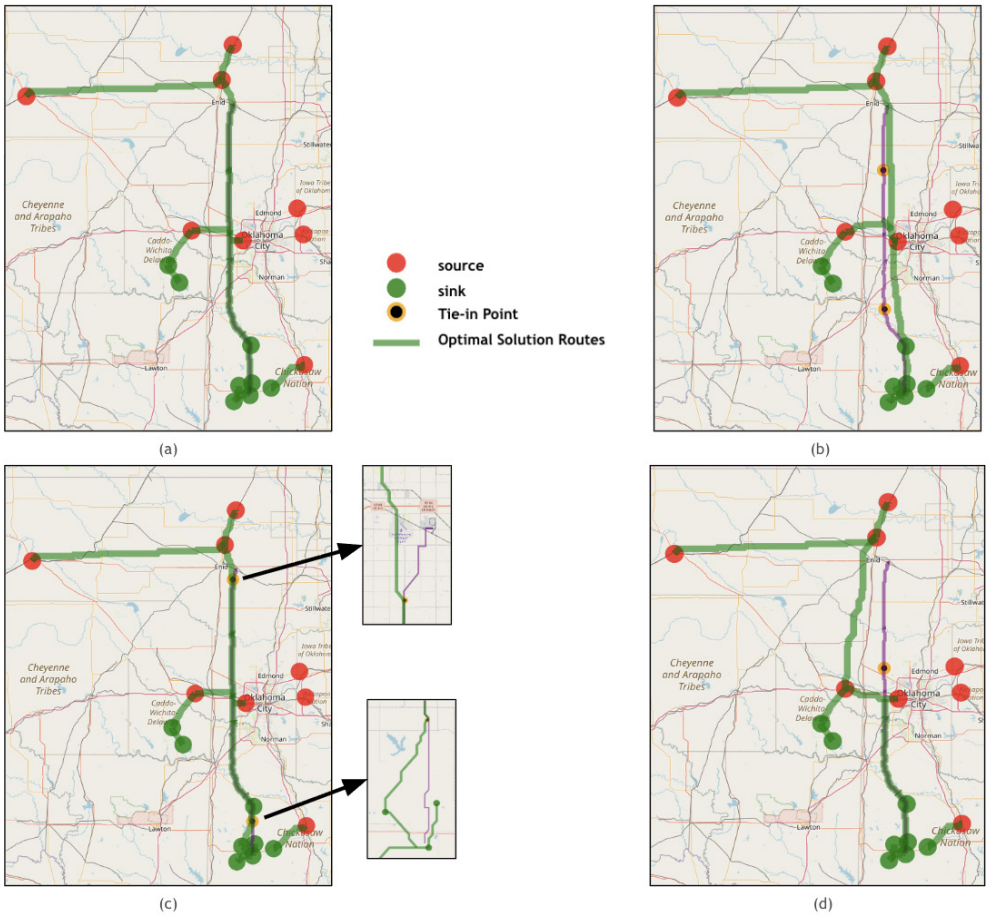

Fig. 8—Sequestrix solutions after embedding the Enid-Purdy pipeline. (a) No tie-in, no exclusion. (b) Two tie-ins, no exclusion. (c) Two tie-ins, exclusion at ends. (d) One tie-in, exclusion before.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

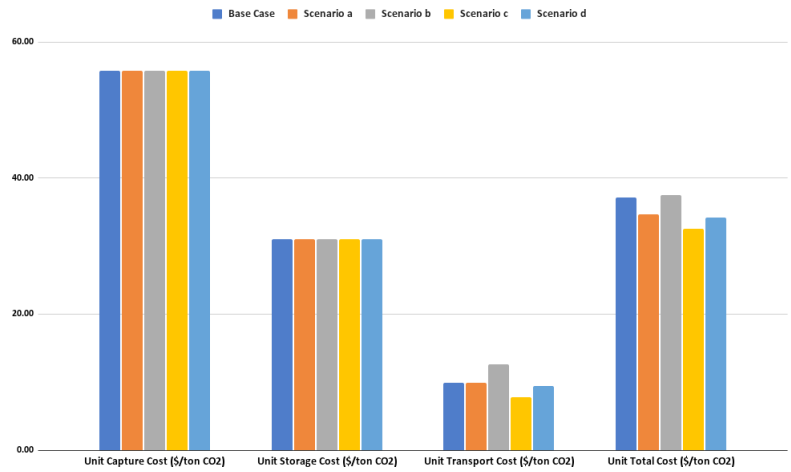

Fig. 9—Unit capture, storage, transport, and total cost differences between the base case and Scenarios a–d.

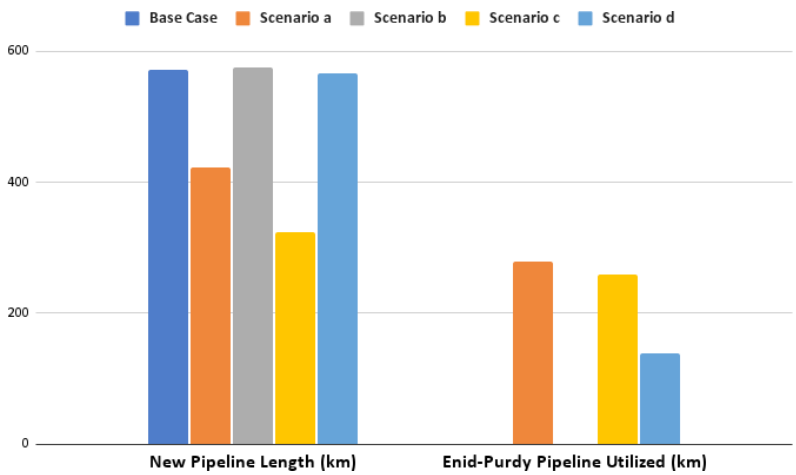

Fig. 10—New and utilized embedded pipeline length differences between the base case and Scenarios a–d.

The process of generating the candidate network is highlighted in **Figs. 7a and 7b** where, first, DT pairs (source-sink, source-source, and sink-sink) are determined and then the LCP is formulated from these pairings. In **Fig. 7c**, the final result of the network optimization is highlighted, and it can be observed that six out of the available eight sources are used to supply captured CO_2 for storage in all eight sinks. The resulting optimal sequestration network requires ~573 km (356 miles) of new pipeline to be built to store the target 1.55 $MTCO_2$ annually with a unit transportation cost of 12.39 USD/tCO_2. The unit cost capture and storage evaluated to be 55.80 USD/tCO_2 and −31 USD/tCO_2, respectively. A comparison of the remaining four scenarios in which the Enid-Purdy pipeline is embedded in the cost surface graph and assigned a build and transport cost of zero is illustrated in **Fig. 8**.

In Scenario a **(Fig. 8a)**, given that there are no tie-in points specified along the Enid-Purdy pipeline, Sequestrix is able to route CO_2 flow through the pipeline up to the maximum capacity of 2 $MTCO_2$, with entry and exit points along the pipeline determined by underlying candidate network and the optimization engine. While the capture and storage costs remain the same (the same sources and sinks in the base case are utilized), the transport cost reduces by 44% to 6.96 USD/tCO_2 by routing CO_2 through 268 km (167 miles) of the Enid-Purdy pipeline, resulting in a 15% decrease in unit cost of sequestration—from 37.15 to 31.72 USD/tCO_2.

In Scenario b **(Fig. 8b)**, the operator specifies only two exact geolocations where entry and exit are permitted along the Enid-Purdy pipeline (highlighted in yellow and black). Due to the discrete nature of these points along the pipeline which serve as the only points of interaction with the existing pipeline, Sequestrix results show that the Enid-Purdy pipeline remains unused. Furthermore, Sequestrix proposes a longer route avoiding the existing pipeline ROW entirely, thus leading to a 2-km increase in new pipeline length with a 2% increase in unit transport cost and 1% increase in overall unit sequestration cost.

Scenario c also provides two tie-in points but allows for entry and exit connections between these two points along the Enid-Purdy pipeline. **Fig. 8c** illustrates that the entry and exit points honor the set limits and the resulting effect is that 258 km (160 miles) of the pipeline is utilized, reducing unit transport cost by 37% when compared with the base case scenario (7.8 USD/tCO_2), and overall unit sequestration cost by 12%.

Finally, in Scenario d, only one tie-in point is specified; however, before the tie-in point along the Enid-Purdy pipeline, no ingress or egress is allowed from the pipeline. Sequestrix simulation results **(Fig. 8d)** show that this constraint is honored, and the pipeline only utilizes 138 km (86 miles) of Enid-Purdy pipeline for transport of CO_2. This results in a 24% reduction in unit transportation costs from base scenario (to 9.43 USD/tCO_2) and an 8% decrease in overall unit sequestration cost.

Comparison metrics for the base case and four other scenarios are illustrated via bar plots in **Figs. 9 and 10**.

Computational Performance and Runtime Summary. Table 4 reports end-to-end wall-clock times for each scenario, along with the core solver time attributable to the MIP solve. Across all cases, the core optimization is a very small fraction of total runtime, while

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

the preprocessing pipeline—loading and weighting the cost-surface graph, DT, candidate-network generation, and (when applicable) encoding of the existing pipeline—dominates runtime. The base case is fastest overall because no existing-pipeline encoding is required. For Scenarios a, c, and d, the Gurobi solve time is lower than the base case because adding the existing pipeline to the candidate network (with the stated constraints) provides cheaper arcs that shrink the effective search space. Scenario b ignores the pipeline during candidate-network generation; its optimization time is therefore similar to the base case (slightly higher), as both must propose new build arcs from scratch.

Scenario	End-to-End (seconds)	Core (seconds)	Preprocessing (seconds)	Core as % of Total
Base case	96.28	0.32	95.96	0.33%
Scenario a	175.88	0.07	175.81	0.04%
Scenario b	190.54	0.39	190.15	0.20%
Scenario c	197.28	0.11	197.17	0.06%
Scenario d	172.57	0.13	172.44	0.08%

Note: Computational environment. Python 3.11; gurobipy 12.0.3; default solver parameters; no time limit. Machine/OS specs: Apple M2 Pro (12-core CPU: 8 performance + 4 efficiency), 32 GB unified memory, macOS 15 (Sequoia).

Table 4—Runtime by scenario. End-to-end = total wall-clock time from data load to solution; Core = Gurobi solve time. Preprocessing = end-to-end – Core.

The optimization itself is fast for the tested instances (≤0.39 seconds; ≤0.33% of total). Differences in end-to-end time are primarily due to preprocessing work. Relative to base, Scenario a achieves the smallest core time by enabling unrestricted ingress/egress on the existing pipeline, biasing the search toward cheaper pipeline arcs. Scenarios c and d also benefit, though the effect is smaller because fewer pipeline segments are usable under their tie-in/exclusion rules. Scenario b's core time is closest to the base case because the existing pipeline is not used during candidate-network generation.

CarbonSAFE II Osage Storage Case Study

Problem Description. The CarbonSAFE Initiative, launched in 2016, aims to bridge critical gaps in CCS deployment by identifying and comprehensively characterizing geological storage sites for CO_2. Through its phased approach encompassing prefeasibility, feasibility, site characterization, permitting, and construction, CarbonSAFE aims to alleviate technical risk and cost and to store more than 50 $MTCO_2$ in commercial-scale saline storage complexes.

As part of the DOE's funding opportunity announcement for Phase II of CarbonSAFE, which focuses on geological storage feasibility projects, research into storing CO_2 in the Arbuckle formation in Osage County was done (Milad et al. 2022). Possible nearby sources of CO_2 and transport infrastructure were identified with the key one being the CVR Partners Coffeyville fertilizer plant in Coffeyville, KS. The CO_2 was to be transported partially using the Coffeyville pipeline (shown in **Fig. 11**) operated by CapturePoint, which currently transports CO_2 to the North Burbank unit in Osage for CO_2-EOR purposes.

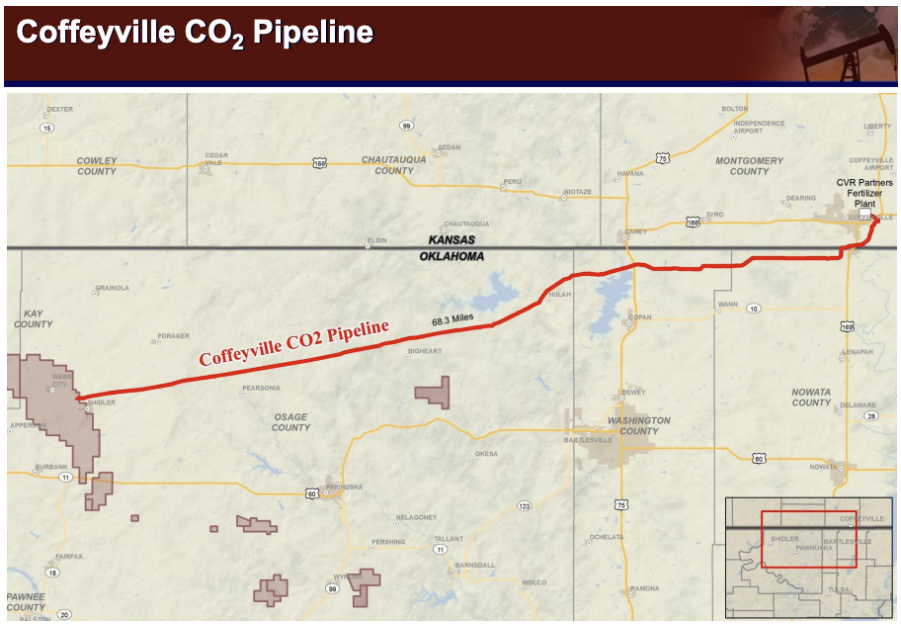

Fig. 11—Map showing CO_2 pipeline from Coffeyville, KS, to the Osage Reservation, OK (from Tracy 2013).

This partial transfer is due to limited additional capacity of the Coffeyville pipeline and therefore, to store the 50 $MTCO_2$ over 30 years as required by the DOE, an additional pipeline construction parallel to the Coffeyville pipeline was proposed. This pipeline takes

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

advantage of existing ROW of the Coffeyville pipeline and will have sufficient transport capacity; however, additional CO_2 sources which can tie-in into this pipeline to deliver CO_2 to the storage site in Osage are required.

Based on the above, the objectives for the technical evaluation were given as follows:

- Estimate construction and operating costs of new pipeline, which will be parallel to the Coffeyville pipeline with the NETL CO_2 transport cost model (CO_2_T_M) tool.
- Assume the geolocation of the Coffeyville pipeline obtained from NPMS to be new pipeline geolocation and use Sequestrix to embed this pipeline with transport cost along the pipeline equal to estimates from above.
- Assess additional nearby CO_2 sources that may be tied into the new pipeline to facilitate required storage and evaluate sequestration costs and economics using 2022 45Q carbon credits.

Estimating New Pipeline Construction Costs. To store 50 $MTCO_2$ over a 30-year period, approximately 1.67 $MTCO_2$ has to be stored annually. Assuming the new pipeline constructed will have to meet flow rate with a capacity factor of 85% (fraction of time when flow is at equivalent of maximum flow rate), the pipeline to be constructed has to have a capacity of ~ 1.96 $MTCO_2$.

The following parameters are utilized as input to the 2022 NETL CO_2 transport cost model (CO_2_T_M):

- The new pipeline length is the same as the existing pipeline length (~68.3 miles)
- Annual CO_2 transported is 1.67 MT/yr
- 85% pipeline capacity factor.
- Project start year of 2024.
- Two-year construction period and 30 years' operation
- Inlet and outlet pressures of 2,200 psig and 1,250 psig, respectively
- Zero elevation.

Table 5 summarizes the result of the model.

Nominal Diameter (in.)	Capital Cost (million USD)	Operating Expense (million USD)	Total Cost (million USD)	Total Volume of CO_2 Transported (MT)	Unit Transport Cost (USD/tCO_2)
8	39.26	90.82	130.08	50	2.60

Table 5—NETL CO_2 transport cost model results for new CapturePoint pipeline.

Embedding New Pipeline, Assessing Additional Sources Tie-In Points and Evaluating Sequestration Economics. To meet Objectives 2 and 3, a simulation was done using Sequestrix with the assumption that CVR Partners could provide an additional 0.72 $MTCO_2$/yr, and that the remaining 0.95 $MTCO_2$/yr would be supplied by neighboring CO_2 emission sources. All this CO_2 will be stored in the Osage North Burbank area in the Arbuckle formation. **Tables 6 and 7** summarize the input data for the simulation.

ID	Unique Name	Capture Capacity ($MTCO_2$/yr)	Total Unit Cost (USD/tCO_2)	Lat.	Long.
1	Northeastern Power Plant	3	75	36.4317	−95.7008
2	Rivertion Power Plant	0.442	75	37.071833	−94.69866
3	Ash-Groove Cement Plant	1.122	56	37.69883	−95.4596

Table 6—CO_2 Sources for Osage CarbonSAFE II study.

ID	Unique Name	Storage Capacity ($MTCO_2$)	Total Unit Cost (USD/tCO_2)	Lat.	Long.
1	Osage North Burbank	28.5	−78	36.796072	−96.67249

Table 7—Osage Arbuckle storage data for the CarbonSAFE II study.

The unit storage cost of −78 USD/tCO_2 was obtained from subtracting the USD 85 45Q tax credits from a unit injection cost of 7/tCO_2.

Using the above data, the results of the Sequestrix simulation are shown in **Fig. 12**.

Sequestrix suggests that the Ash-Groove cement plant, KS, is the most optimal (cost-effective) CO_2 source that can tie into the new pipeline to transport CO_2 to the Osage North Burbank storage area. Combining Sequestrix's result with the knowledge of the additional CO_2 being supplied by CVR Partners, the project economics can be summarized as follows:

- An 84-km pipeline will be built from Ash-Groove cement plant to new pipeline inlet point located in Coffeyville, KS, USA, to transport 0.95 MT of captured CO_2 annually from the cement plant.
- The 0.95 MT plus an additional 0.72 MT of CO_2 from CVR Partners (total of 1.67 $MTCO_2$) will be transported annually along the new pipeline from Coffeyville, KS to Osage North Burbank area, OK.
- The total unit transport cost from these operations is 10.18 USD/tCO_2 for the 1.67 $MTCO_2$ transported annually.
- The average unit capture cost from both facilities of 39.19 USD/tCO_2 combined with unit storage costs of −78 USD/tCO_2 and transportation cost listed above brings the total unit cost to −28.63 USD/tCO_2.

Utilizing Sequestrix, CapturePoint pipeline specs, and CO_2_T_M, a technical evaluation of the proposed CarbonSAFE II project was done and shown to generate net revenue for every ton of CO_2 stored over the 30-year duration.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

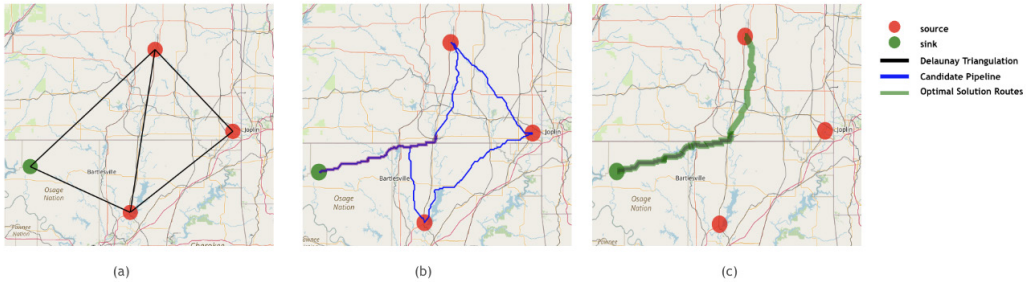

Fig. 12—Sequestrix solution for the Osage CarbonSAFE II Study. (a) DT results. (b) Alternate pipeline routes. (c) Optimal pipeline path selected.

Conclusion

The aim of this study was to present a method for embedding existing pipelines into CO_2 sequestration network optimization. An algorithm was presented to achieve this embedding in the cost surface graph prior formulation of the MILP problem for optimization. Further algorithms to manage ingress and egress into these embedded pipelines are presented in Appendix A for reference, and the entire solution was packaged into an open-source sequestration optimization tool called Sequestrix, which builds on some defined algorithms from SimCCS.

Using Sequestrix, an example of embedding the Enid-Purdy pipeline was illustrated, and we showed that cost savings occurring from tying into existing pipelines can be evaluated. In our examples, the transportation cost could be reduced by as much as 44% if there is no restriction on entry and exit on the embedded pipeline. However, if discrete tie-in points are defined, a suboptimal solution that avoids the pipeline entirely may also emerge, leading to an increase in transportation costs. In the final example, we showed that Sequestrix can be used in combination with other pipeline costing tools, proprietary data, and preferred sources to find optimal tie-in points for defined pipeline routes. This aided the evaluation of a proposed CarbonSAFE II sequestration project in Osage County, OK.

CCS researchers and industry professionals may access Sequestrix via the GitHub page and run simulations locally on the web-based interface or deploy Sequestrix to internal servers for more distributed access. Sequestrix can also be applied to hydrogen storage assessment given that the problems are similarly phrased. Once the geolocation hydrogen production plants and proposed storage hubs are known, transport routes may be generated with inclusion of existing infrastructure for robust economic analysis.

Extending Work: Multiperiod (Time-Varying) Planning. The cases presented here assume a constant annual capture target for the planning horizon. The formulation can be extended to multiperiod operations by indexing flows and limits by planning period t (e.g., year) while retaining the same sets and symbols. In practice, source capture a_i, sink injection b_j, and arc flows f_{ac} become $a_{i,t}$, $b_{j,t}$, and $f_{ac,t}$; source capture limits and sink injectivity become $Q^S_{i,t}$ and $Q^R_{j,t}$. The objective keeps the same cost components (capture, storage, transport, and build) and applies them per period; project economics may be evaluated with the same capital-recovery treatment and, if desired, discounting across t. The constraint set is enforced for each period (capacity on arcs, flow balance at nodes, and capture/injectivity bounds), and the target may be imposed as either a per-period trajectory $\sum_{i\in S} a_{i,t}$ or a cumulative requirement $\sum_t \sum_{i\in S} a_{i,t}$. Optional ramp limits on $a_{i,t}$, $b_{j,t}$, or $f_{ac,t}$ can bound fluctuations to reflect operational realities. A compact, notation-consistent statement of this extension is provided in Appendix C.

Nomenclature

A = set of all node-to-node arcs representing alternate and existing pipelines
C = set of pipeline trends which is a linear representation of cost (USD million/km) vs. volume of CO_2 transported ($MTCO_2$)
I = set of all nodes
N = set of all transshipment nodes
$P \epsilon A$ = set of all existing pipeline arcs
R = set of all CO_2 sinks or injection sites nodes
S = set of all CO_2 capture source nodes

Parameters

F^S_i = fixed capture cost of CO_2 at source i, USD million
F^R_j = fixed storage cost of CO_2 at sink j, USD million
Q^{max}_{ac} = maximum annual capacity of pipeline arc a with trend c, $MTCO_2$/yr
Q^{min}_{ac} = minimum annual capacity of pipeline arc a with trend c, $MTCO_2$/yr
Q^S_i = CO_2 annual capture capacity at source i, $MTCO_2$/yr
Q^R_j = CO_2 storage capacity at sink j, $MtCO_2$
V^S_i = variable capture cost of CO_2 at source i, USD/tCO_2
V^R_j = variable capture cost of CO_2 at sink j, USD/tCO_2
α_{ac} = transportation cost of pipeline arc a with trend c, USD/tCO_2
β_{ac} = build cost of pipeline arc a with trend c, USD million/yr

Decision Variables

$a_i \in \mathbb{R}$ = rate of CO_2 captured at source i, tCO_2/yr
$b_j \in \mathbb{R}$ = rate of CO_2 stored at sink j, tCO_2/yr

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

$CO_2T \in R$ = target amount of CO_2 to be sequestered during project life
$f_{ac} \in \mathbb{R}$ = amount of CO_2 flow in the pipeline arc is built with trend c, tCO_2/yr
$r_j \in \{0, 1\}$ = binary variable indicating if a sink j is activated
$s_i \in \{0, 1\}$ = binary variable indicating if a source i is activated
$y_{ac} \in \{0, 1\}$ = binary variable indicating if a pipeline arc a with trend c is built

Acknowledgments

We would like to acknowledge and appreciate the Carbon Utilization Storage Partnership of Western United States (CUSP West) for their contribution to our research. We also appreciate CapturePoint Solutions LLC for sharing valuable knowledge with us during this study.

References

Abramson, E., McFarlane, D., and Brown, J. 2020. *Transport Infrastructure for Carbon Capture and Storage: Regional Infrastructure for Midcentury Decarbonization*. Minneapolis, Minnesota, USA: Great Plains Institute. https://www.betterenergy.org/wp-content/uploads/2020/06/GPI_Regional CO2Whitepaper.pdf.

CEPS. 2010. *CO_2 Highways for Europe: Modelling a Carbon Capture, Transport and Storage Infrastructure for Europe*. https://www.ceps.eu/ceps-publications/co2-highways-europe-modelling-carbon-capture-transport-and-storage-infrastructure/.

Chen, B., Ma, Z., and Pawar, R. 2022. *O_2 Transport Infrastructure Modeling in the Intermountain WEST Region, USA. SSRN Preprint (I-WEST)*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5021131.

Congressional Research Service. 2024. *Siting Challenges for Carbon Dioxide (CO_2) Pipelines*. https://www.congress.gov/crs_external_products/IN/PDF/IN12269/IN12269.4.pdf.

DaneshFar, J., Nnamdi, D., Moghanloo, R. G. et al. 2021. Economic Evaluation of CO2 Capture, Transportation, and Storage Potentials in Oklahoma. Paper presented at the SPE Annual Technical Conference and Exhibition, Dubai, UAE, 21−23 September. https://doi.org/10.2118/206106-MS.

Delaunay, B. 1934. Sur La Sphère Vide (On the Empty Sphere). À La Mémoire de Georges Voronoï. In *Bulletin de l'Académie Des Sciences de l'URSS, Classe Des Sciences Mathématiques et Naturelles*, Vol. 6, 793–800.

Dijkstra, E. W. 1959. A Note on Two Problems in Connexion with Graphs. *Numer Math* **1** (1): 269–271. https://doi.org/10.1007/BF01386390.

DOE/NETL. 2025. *CarbonSAFE Initiative*. https://www.netl.doe.gov/carbon-management/carbon-storage/carbonsafe (accessed 10 September 2025).

Gale, J., Christensen, N. P., Cutler, A. et al. 2001. Demonstrating the Potential for Geological Storage of CO_2: The Sleipner and GESTCO Projects. *Environ Geosci* **8** (1): 160–165. https://doi.org/10.1046/j.1526-0984.2001.008003160.x.

Hamzat, A., Milad, B., Moghanloo, R. et al. 2022. Flow Simulation of CO_2 Plume in Arbuckle Group. Paper presented at the AGU Fall Meeting 2022, Chicago, Illinois, USA, 12−16 December.

Hoover, B., Yaw, S., and Middleton, R. S. 2020. *CostMAP*: An Open-Source Software Package for Developing Cost Surfaces Using a Multi-Scale Search Kernel. *Int J Geogr Inf Sci* **34** (3): 520–538. https://doi.org/10.1080/13658816.2019.1675885.

Jones, E. C., Yaw, S., Bennett, J. A. et al. 2022. Designing Multi-Phased CO_2 Capture and Storage Infrastructure Deployments. *Renew Sustain Energy Transit* **2**: 100023. https://doi.org/10.1016/j.rset.2022.100023.

Keil, J. M. and Gutwin, C. A. 1992. Classes of Graphs Which Approximate the Complete Euclidean Graph. *Discrete Comput Geom* **7** (1): 13–28. https://doi.org/10.1007/BF02187821.

Ma, Z., Chen, B., and Pawar, R. J. 2023. Phase-Based Design of CO_2 Capture, Transport, and Storage Infrastructure via SimCCS$^{3.0}$. *Sci Rep* **13**: 6527. https://www.nature.com/articles/s41598-023-33512-5.

Ma, Z., Chen, B., and Pawar, R. 2024. Reuse of Existing CO_2 Pipeline and Pipeline Rights-of-Way for Large-Scale CCS Deployments. Paper presented at the SPE Annual Technical Conference and Exhibition, New Orleans, Louisiana, USA, 23−25 September. https://doi.org/10.2118/220930-MS.

McPherson, B. J. O. L. and Cole, B. S. 2000. Multiphase CO_2 Flow, Transport and Sequestration in the Powder River Basin, Wyoming, USA. *J Geochem Explor* **69–70**: 65–69. https://doi.org/10.1016/S0375-6742(00)00046-7.

Middleton, R. S. 2013. A New Optimization Approach to Energy Network Modeling: Anthropogenic CO_2 Capture Coupled with Enhanced Oil Recovery. *Int J Energy Res* **37** (14): 1794–1810. https://doi.org/10.1002/er.2993.

Middleton, R. S. and Bielicki, J. M. 2009. A Scalable Infrastructure Model for Carbon Capture and Storage: SimCCS. *Energy Policy* **37** (3): 1052–1060. https://doi.org/10.1016/j.enpol.2008.09.049.

Middleton, R. S., Yaw, S. P., Hoover, B. A. et al. 2020. SimCCS: An Open-Source Tool for Optimizing CO_2 Capture, Transport, and Storage Infrastructure. *Environ Model Softw* **124**: 104560. https://doi.org/10.1016/j.envsoft.2019.104560.

Milad, B., Daneshfar, J., and Moghanloo, R. G. 2022. CO_2 Storage Potential in Arbuckle Reservoir: A Case Study in Osage County, Oklahoma. Paper presented at the Energy in Data Conference, Austin, Texas, USA, 20−23 February. https://doi.org/10.7462/eid2022-07.1.

Morbee, J., Serpa, J., and Tzimas, E. 2011. Optimal Planning of CO_2 Transmission Infrastructure: The JRC InfraCCS Tool. *Energy Procedia* **4**: 2772–2777. https://doi.org/10.1016/j.egypro.2011.02.180.

Middleton, R. S., Kuby, M. J., and Bielicki, J. M. 2012. Generating Candidate Networks for Optimization: The CO_2 Capture and Storage Optimization Problem. *Comput Environ Urban Syst* **36** (1): 18–29. https://doi.org/10.1016/j.compenvurbsys.2011.08.002.

Shih, J.-S., Chen, B., Thompson, A. et al. 2023. Modeling Ecological Constraints on a CO_2 Pipeline Network. *Environ Sci Technol* **57** (43). https://doi.org/10.1021/acs.est.3c01721.

Talsma, C., Middleton, E., and Middleton, R. 2022. CostmapPro: Addressing the Massive-Scale CO2 Pipeline Challenge. *Proceedings of the 16th Greenhouse Gas Control Technologies Conference (GHGT-16)*. https://doi.org/10.2139/ssrn.4273192.

Tracy, K. 2013. A CO2 Midstream Overview. Presented at the CO_2 Conference. https://www.co2conference.net/wp-content/uploads/2014/01/13-Chaparral-CO2-Midstream-Overview-2013.12.09new.pdf (accessed 27 April 2023).

Velasco-Lozano, M., Ma, Z., Chen, B. et al. 2024. Optimizing Large-Scale CO_2 Pipeline Networks Using a Geospatial Splitting Approach. *J Environ Manage* **370**: 122522. https://doi.org/10.1016/j.jenvman.2024.122522.

Wallace, M., Goudarzi, L., Callahan, K. et al. 2015. *A Review of the CO□ Pipeline Infrastructure in the U.S. DOE/NETL-2014/1681*. Pittsburgh, Pennsylvania, USA: National Energy Technology Laboratory (NETL). https://doi.org/10.2172/1487233.

Whitman, C., Yaw, S., Hoover, B. et al. 2022. Scalable Algorithms for Designing CO_2 Capture and Storage Infrastructure. *Optim Eng* **23** (2): 1057–1083. https://doi.org/10.1007/s11081-021-09621-3.

Yaw, S., Middleton, R. S., and Hoover, B. 2019. Graph Simplification for Infrastructure Network Design. In *Combinatorial Optimization and Applications*, eds. Y. Li, M. Cardei, and Y. Huang, Vol. 11949, 576–589. Cham, Switzerland: Springer International Publishing. https://doi.org/10.1007/978-3-030-36412-0_47.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

Appendix A: Algorithms for Controlling Entry and Exit on Embedded Pipelines

Algorithm A-1—Case 1.

```
Input: G(V, E) a cost surface graph; P(V), a list of graph vertices along an existing pipeline; tie_points, a list containing
geolocation of two preferred tie-in points on existing pipeline
Output: G'(V, E'), a modified cost surface graph with some edge weights along pipeline vertices set to 1e9
Create A list: tie_vertices = list()
forall nodepair in tie_points:
    cell = findCell(nodepair)
    tie_vertices.append(cell)
end for
forall edges in G.edges:
    #in
    if (edges[1] in P) and (edges[0] not in P) and (edges[1] != tie_vertices[0]) and (edges[1] != tie_vertices[1]):
        G.edges[edges][weight] =1e9
    end if
    #out
    if (edges[0] in P) and (edges[1] not in P) and (edges[0] !+ tie_vertices[0]) and (edges[0] != tie_vertices[1]):
    G.edges[edges][weight] =1e9
    end if
end for
```

Algorithm A-2—Case 2.

```
Input: G(V, E) a cost surface graph; P(V), a list of graph vertices along an existing pipeline; tie_points, a list containing
geolocation of two preferred tie-in points on existing pipeline
Output: G'(V, E'), a modified cost surface graph with some edge weights along pipeline vertices set to 1e9
Create A list: tie_vertices = list()
forall nodepair in tie_points:
    cell = findCell(nodepair)
    tie_vertices.append(cell)
end for
Create A list: exclusion = list()
idx_1 = P.index(tie_vertices[0]) #index location of point1
idx_2 = P.index(tie_vertices[1]) #index location of point2
exclusion = P[:idx_1] +P[idx_2:] #slice P and get points before and after tie points on both sides
forall edges in G.edges:
    #in
    if (edges[1] in exclusion) and (edges[0] not in P) and (edges[1] != tie_vertices[0]) and (edges[1] != tie_vertices[1]):
        G.edges[edges][weight] =1e9
    end if
    #out
    if (edges[0] in exclusion) and (edges[1] not in P) and (edges[0] != tie_vertices[0]) and (edges[0] != tie_vertices[1]):
        G.edges[edges][weight] =1e9
    end if
end for
```


Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

Algorithm A-3—Case 3.

```
Input: G(V, E) a cost surface graph; P(V), a list of graph vertices along an existing pipeline; tie_points, a list containing
geolocation of one preferred tie-in point on existing pipeline
Output: G'(V, E'), a modified cost surface graph with some edge weights along pipeline vertices set to 1e9
Create A list: tie_vertices = list()
forall nodepair in tie_points:
    cell = findCell(nodepair)
    tie_vertices.append(cell)
end for
Create A list: exclusion = list()
exclusion = P[:−1] #assuming source/sink is at end of pipeline OR
exclusion = P[1:] #assuming source/sink is at beginning of pipeline
forall edges in G.edges:
    #in
    if (edges[1] in exclusion) and (edges[0] not in P) and (edges[1] != tie_vertices[0]) and (edges[1] != tie_vertices[1]):
        G.edges[edges][weight] =1e9
    end if
    #out
    if (edges[0] in exclusion) and (edges[1] not in P) and (edges[0] != tie_vertices[0]) and (edges[0] != tie_vertices[1]):
        G.edges[edges][weight] =1e9
    end if
end for
```

Algorithm A-4—Case 4.

```
Input: G(V, E) a cost surface graph; P(V), a list of graph vertices along an existing pipeline; tie_points, a list containing
geolocation of one preferred tie-in point on existing pipeline
Output: G'(V, E'), a modified cost surface graph with some edge weights along pipeline vertices set to 1e9
Create A list: tie_vertices = list()
forall nodepair in tie_points:
    cell = findCell(nodepair)
    tie_vertices.append(cell)
end for
Create A list: exclusion = list()
exclusion = P[:P.index(tie_vertices[0])] #assuming exclusion zone is before tie-in point OR
exclusion = P[P.index(tie_vertices[0])+1:] #assuming exclusion zone is after tie-in point
forall edges in G.edges:
    #in
    if (edges[1] in exclusion) and (edges[0] not in P) and (edges[1] != tie_vertices[0]) and (edges[1] != tie_vertices[1]):
        G.edges[edges][weight] =1e9
    end if
    #out
    if (edges[0] in exclusion) and (edges[1] not in P) and (edges[0] != tie_vertices[0]) and (edges[0] != tie_vertices[1]):
        G.edges[edges][weight] =1e9
    end if
end for
```

Appendix B: Optimization Settings and Convergence Criteria in Sequestrix

We solved models with Gurobi as the primary solver and set CPLEX (via PuLP) as a fallback for very large instances in Sequestrix. We did not set custom convergence criteria; we used solver defaults:

- Gurobi: default relative MIP optimality gap = 1e−4 (0.01%), standard feasibility/integrality tolerances, no time limit.
- CPLEX (via PuLP): default relative MIP gap = 1e−4, standard feasibility/integrality tolerances, no time limit.

No early-stopping rules (node/solution limits) were used; all other parameters remained at solver defaults.

Appendix C: Multiperiod extension

Let $T = \{1, \ldots, T\}$ be index planning periods (e.g., years). The existing sets S, R, N, I, C, A, P, parameters $\left(Q_i^S,\ Q_j^R,\ Q_{ac}^{max},\ Q_{ac}^{min},\ \alpha_{ac}\ , \beta_{ac}\right)$, and decisions $\left(s_i,\ r_j\ ,\ y_{ac},\ a_i,\ b_j, f_{ac}\ \right)$ are retained from the main formulation. Only the flows are time-indexed; activation/build variables remain time-invariant.

Time-Indexed Data (Per Year). $Q_{i,t}^S$ = annual capture upper bound at source i in period t ($MTCO_2$/yr).

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025

$Q_{j,t}^{R}$ = annual injectivity limit at sink j in period t ($MTCO_2$/yr). (*If a project-life storage cap is also required, retain Q_j^R as a cumulative bound; see constraint below.*)

Time-Indexed Decisions (Flows).

- $a_{i,t} \geq 0$, $b_{j,t} \geq 0$, $f_{ac,t} \geq 0$ are the period- t analogs of a_i, b_j, and f_{ac}.

Objective (Same Cost Components as the Main Model, Applied Per Period).

$$\min \sum_{t \in T} \left[\sum_{i \in S} \left(V_i^S a_{i,t} \right) + \sum_{j \in R} \left(V_j^R b_{j,t} \right) + \sum_{a \in A} \sum_{c \in C} \left(\alpha_{ac} f_{ac,t} + \beta_{ac} y_{ac,} \right) \right] + \sum_{i \in S} \left(F_i^S s_i \right) + \sum_{j \in R} \left(F_j^R r_j \right).$$

with the same capture, storage, pipeline-build, and transport cost terms as in the single-period model.

Constraints (Enforced for Each t).

- Arc capacity: $Q_{ac}^{min} \leq f_{ac,t} \leq Q_{ac}^{max} \quad \forall\, a \in A,\ \forall\, c \in C,\ \forall\, t \in T.$
- Flow balance at nodes (sources/sinks/trans-shipment) as in the main model, now for each t.
- Demand/supply balance at sinks/sources: as in the main model, now for each t.
- Capture bounds: $0 \leq a_{i,t} \leq Q_i^S s_i \quad \forall i \in S,\ \forall t \in T.$
- Injectivity bounds: $0 \leq b_{j,t} \leq Q_j^R r_j \quad \forall j \in R,\ \forall t \in T.$
- Cumulative storage bound: $\sum_{t \in T} b_{j,t} \leq Q_j^R r_j \quad \forall j \in R.$
- Target options**:**
  Cumulative $\sum_{t \in T} \sum_{i \in S} a_{i,t} \geq CO_2 T$ (direct multiperiod analog of the single-period target).
  Per-period trajectory (optional): specify $CO_2 T_t$ and enforce $\sum_{i \in S} a_{i,t} \geq CO_2 T_t \ \forall t \in T.$

Activation $(s_i,\ r_j)$ and build (y_{ac}) decisions remain one-time (time-invariant), matching the current annualized treatment of β_{ac}. If staged expansion is desired in future work, these binaries can be time-indexed, but that would introduce new variables and is not included here.

Downloaded from http://onepetro.org/SJ/article-pdf/doi/10.2118/214917-PA/5290531/spe-214917-pa.pdf/1 by David Nnamdi on 04 November 2025